

1. CPU / Non-CUDA Execution Provider Operator

Any operator assigned to the CPUExecutionProvider or another non-CUDA execution provider becomes a hard CUDA Graph boundary. CUDA Graph capture can only include CUDA work, so the CUDA regions before and after the CPU operator must be captured separately.

2. Control-Flow Operator

Operators such as If, Loop, and Scan can change the sequence of operations executed at runtime. Since CUDA Graph replay assumes a predictable execution sequence, these operators are typically treated as boundaries and executed eagerly, while CUDA regions around them can still be captured.

3. Value-Dependent Dynamic Output Operator

Operators such as NonZero, Unique, Compress, or NonMaxSuppression may produce output sizes that depend on the actual input values rather than only on the input shape. Since downstream memory requirements and kernel configurations may therefore be unknown until runtime, these operators are strong candidates for graph boundaries.

4. CUDA Operator with Host-Dependent Behaviour

Some operators may execute on the GPU but still require CPU-side synchronization, inspect GPU-generated values on the host, or make runtime decisions before launching subsequent kernels. Such operators cannot always be safely replayed as part of a fixed CUDA Graph and may therefore create a boundary.

5. Capture-Unsafe CUDA Operator

A CUDA operator may use CUDA APIs, libraries, synchronization mechanisms, or runtime behaviour that is incompatible with CUDA stream capture. Even though the operator runs on the CUDA Execution Provider, it may still need to execute outside the captured region.

6. Operator with Unstable or Unpredictable Memory Allocation

Some operators require temporary buffers or workspaces whose size or memory address changes significantly between executions. Since CUDA Graph replay relies heavily on stable memory behaviour, such operators may become boundaries unless a fixed or stable-address memory arena can be provided.

7. Dynamic-Shape CUDA Operator

A normal dynamic shape is NOT automatically a graph boundary. If the shape is known before inference begins, the operator can remain inside the CUDA region and a separate CUDA Graph can be captured for each shape or shape class, such as G32, G64, and G128.

8. Data-Dependent Control or Variable Kernel Sequence

If an operator causes a different number or ordering of GPU kernels to execute depending on runtime data, the captured GPU execution sequence may no longer be reusable. Such behaviour generally requires either specialization for each execution pattern or a graph boundary.

9. Execution Provider Transition

A transition such as CUDA → CPU, CUDA → TensorRT, or CUDA → another accelerator provider naturally separates CUDA Graph regions. Each CUDA-owned section can potentially be captured independently, while the provider transition is handled normally by ONNX Runtime.

10. Very Small CUDA Region as a Profitability Boundary

A region may be technically capturable but contain too few kernels for CUDA Graph replay to provide a meaningful latency improvement. In this case, the runtime can deliberately leave the region in normal eager execution instead of creating and storing another CUDA Graph.